

Билет 16. Stackful и stackless корутины. C++ coroutines, `co_await`, `co_return`

0. Формулировка билета

Stackful и stackless/stateless корутины. Сходства и различия с файберами. Корутины в C++, их внутреннее устройство в общих чертах. Оператор `co_await`. Ключевое слово `co_return`.

Главная идея:

Корутина — это обобщение обычной функции.
Обычная функция вошла один раз и вышла один раз.
Корутина может приостановиться в середине и потом продолжиться с того же места.

Обычная функция:

```
call
    выполняется от начала до конца
return
```

Корутина:

```
resume
    выполняется до suspend point
suspend
resume
    продолжает после suspend point
suspend / return
```

Ключевые слова:

```
suspend — приостановить выполнение
resume  — продолжить выполнение
```

1. Корутина как идея

Обычная подпрограмма:

```
int f() {  
    int x = 1;  
    int y = 2;  
    return x + y;  
}
```

У неё простой control flow:

вошли в начало функции
выполнили код
вышли через return

Корутина может сделать иначе:

вошли
сделали часть работы
приостановились
вернулись к вызывающему
потом продолжили с того же места

Пример идеи:

```
resume coroutine  
    print "step 1"  
    suspend  
  
resume coroutine  
    print "step 2"  
    suspend  
  
resume coroutine  
    print "step 3"  
    finish
```

То есть корутина хранит своё состояние между вызовами `resume`.

2. Зачем нужны корутины

Корутины нужны, чтобы писать код в естественном последовательном стиле, но при этом позволять приостанавливать выполнение.

Пример 1. Генератор

Хотим писать:

```
Generator<int> Numbers() {  
    co_yield 1;  
    co_yield 2;  
    co_yield 3;  
}
```

А использовать:

```
for (int x : Numbers()) {  
    std::cout << x << "\n";  
}
```

Смысл:

функция не возвращает все значения сразу
она отдаёт по одному значению и приостанавливается

Пример 2. Асинхронный код

Без корутин:

```
ReadAsync(fd, [](Request req) {  
    ProcessAsync(req, [](Response resp) {  
        WriteAsync(fd, resp, [] {  
            // done  
        });  
    });  
});
```

С корутинами хочется писать:

```
Task<void> Session(Socket s) {  
    Request req = co_await ReadAsync(s);  
    Response resp = co_await ProcessAsync(req);  
    co_await WriteAsync(s, resp);  
}
```

Код выглядит почти синхронно:

```
прочитал  
обработал  
записал
```

Но под капотом каждая операция может приостанавливать корутину и возвращать управление event loop-у.

3. Stackful корутины

Stackful coroutine — корутина со своим стеком.

Это очень похоже на файбер.

У неё есть:

свой стек
сохранённый stack pointer
сохранённые регистры
возможность остановиться глубоко внутри цепочки вызовов

Например:

```
void Session() {  
    ReadRequest();  
}  
  
void ReadRequest() {  
    ParseHeader();  
}  
  
void ParseHeader() {  
    suspend();  
}
```

Если корутина stackful, то `suspend()` может произойти глубоко внутри `ParseHeader`, и весь стек вызовов сохранится:

Session
 ReadRequest
 ParseHeader
 suspend

При `resume` выполнение продолжится внутри `ParseHeader`.

4. Файбер как пример stackful coroutine

Файбер можно рассматривать как разновидность stackful coroutine + scheduler.

Файбер имеет:

```
свой стек  
context switch  
yield/suspend/resume  
scheduler
```

Поэтому:

```
fiber ≈ stackful coroutine, которую планирует runtime
```

Но есть нюанс:

Корутина как идея — это только механизм “приостановиться и продолжиться”.
Файбер — это уже часть runtime: scheduler, очереди runnable fibers, ожидание I/O, timers, fiber-aware mutex.

То есть:

```
stackful coroutine:  
    механизм сохранения стека и продолжения  
  
fiber:  
    stackful coroutine + scheduler/runtime
```

5. Stackless корутины

Stackless coroutine — корутина без собственного отдельного стека.

В C++20 корутины именно stackless.

Это значит:

```
корутина не сохраняет весь стек вызовов  
компилятор превращает одну coroutine-функцию в state machine  
локальные переменные, переживающие suspend, выносятся в coroutine frame
```

Если обычная функция была такой:

```
Task<void> Foo() {  
    int x = 1;  
    co_await Something();  
    int y = x + 1;
```

```
    co_return;  
}
```

то компилятор примерно делает из неё автомат:

```
state = 0:  
    x = 1  
    начать Something  
    suspend  
  
state = 1:  
    продолжить после co_await  
    y = x + 1  
    finish
```

`x` должен пережить `co_await`, поэтому он хранится не на обычном стеке вызова, а в coroutine frame.

6. Stackful vs stackless

Свойство	Stackful	Stackless
Свой стек	Да	Нет отдельного полного стека
Кто сохраняет состояние	Runtime / context switch	Компилятор
Где можно остановиться	Почти где угодно глубоко в вызовах	Только в функции с <code>co_await/</code> <code>co_yield/co_return</code>
Цена	Нужно выделять стек	Обычно дешевле
Код	Меньше “швов”	Нужны <code>co_await</code> в точках suspend
Пример	Fiber	C++20 coroutine

Главная разница:

```
stackful:  
    сохраняет стек целиком  
  
stackless:  
    сохраняет только состояние конкретной coroutine-функции в coroutine frame
```

7. Почему stackless-корутина не может остановиться “где угодно”

Допустим:

```
Task<void> A() {  
    B();  
}  
  
void B() {  
    C();  
}  
  
void C() {  
    // хотим suspend  
}
```

В stackful-файбере это возможно: `C` может вызвать `suspend`, и весь стек `A → B → C` сохранится.

В stackless C++ coroutine так нельзя, если `B` и `C` — обычные функции.

Чтобы `suspend` прошёл через цепочку, функции должны быть корутинами и явно использовать `co_await`:

```
Task<void> A() {  
    co_await B();  
}  
  
Task<void> B() {  
    co_await C();  
}  
  
Task<void> C() {  
    co_await Something();  
}
```

То есть stackless-корутины требуют “швов” в коде:

```
co_await  
co_return  
co_yield
```

8. Почему stackless-корутины дешевле

Stackful-файберу нужен стек.

Даже если стек выделяется лениво, runtime всё равно должен думать про:

```
память под стек  
guard pages  
context switch  
sanitizer integration  
переполнение стека
```

Stackless-корутина хранит только то, что реально нужно пережить suspend.

Например:

```
Task<int> Foo() {  
    int a = 1;  
    int b = co_await Bar();  
    co_return a + b;  
}
```

В coroutine frame попадут:

```
state  
a  
место для результата Bar  
promise object  
awaiter/temporary objects, если нужны
```

То есть frame может быть намного меньше полного стека.

9. Coroutine frame

Когда вызывается C++ coroutine, создаётся **coroutine frame**.

В нём лежит:

```
состояние автомата  
локальные переменные, живущие через suspend  
параметры функции, если нужны после suspend  
promise object
```



```
текущий awaiter  
служебная информация для resume/destroy
```

Обычная функция хранит локальные переменные на стеке.

Корутина, которая может приостановиться, не может хранить всё только на стеке вызывающего, потому что вызывающая функция может уже вернуться.

Поэтому:

```
данные, нужные после suspend, живут в coroutine frame
```

Frame обычно размещается в куче, но компилятор может оптимизировать аллокацию, если видит, что это безопасно.

10. Coroutine handle

У корутины есть handle:

```
std::coroutine_handle<>
```

Он позволяет runtime-у управлять корутиной:

```
handle.resume();  
handle.destroy();  
handle.done();
```

Смысл:

```
resume — продолжить выполнение корутины  
destroy — уничтожить coroutine frame  
done — проверить, завершена ли корутина
```

Если корутина приостановилась на `co_await`, кто-то должен сохранить её handle и потом вызвать `resume`.

Например:

```
co_await ReadAsync(fd)  
↓  
корутина приостановилась  
event loop сохранил handle
```

```
fd стал готов  
event loop вызывает handle.resume()
```

11. Return object

C++ coroutine не возвращает обычный результат напрямую.

Например:

```
Task<int> Foo() {  
    co_return 42;  
}
```

Вызов `Foo()` обычно не выполняет всю функцию до конца.

Он создаёт coroutine frame и возвращает объект:

```
Task<int>
```

Этот объект — handle/обёртка для асинхронного результата.

То есть:

```
Foo() возвращает Task<int>  
а не сразу int
```

Потом этот `Task<int>` можно:

```
co_await Foo();
```

или запустить в scheduler-е.

12. `promise_type`

У каждой C++ coroutine есть promise object.

Важно:

`promise_type` в C++ coroutine — это не то же самое, что `std::promise`.

Это служебный объект, через который компилятор настраивает поведение корутины.

Если функция возвращает `Task<T>`, то компилятор ищет:

```
Task<T>::promise_type
```

В `promise_type` могут быть методы:

```
get_return_object()
initial_suspend()
final_suspend()
return_value(...)
return_void()
unhandled_exception()
```

Смысл:

```
get_return_object – что вернуть вызывающему при создании корутины
initial_suspend   – надо ли остановиться сразу в начале
final_suspend     – что сделать при завершении
return_value      – что делать с co_return value
return_void       – что делать с co_return без значения
unhandled_exception – куда положить необработанное исключение
```

13. Схема запуска C++ coroutine

Для функции:

```
Task<int> Foo() {
    co_return 42;
}
```

Компилятор примерно генерирует такую логику:

1. Создать coroutine frame.
2. Создать promise object внутри frame.
3. Получить return object через `promise.get_return_object()`.
4. Выполнить `co_await promise.initial_suspend()`.
5. Выполнить тело корутины.
6. Если встретился `co_return 42`:
 `promise.return_value(42)`
7. Если вылетело исключение:
 `promise.unhandled_exception()`

8. Выполнить `co_await promise.final_suspend()`.
9. Вернуть управление вызывающему/runtime-y.

`Task<int>` обычно хранит `coroutine_handle<promise_type>`.

14. `co_return`

`co_return` завершает корутину.

В обычной функции:

```
return 42;
```

значит:

вернуть 42 вызывающему

В корутине:

```
co_return 42;
```

значит примерно:

```
promise.return_value(42)  
перейти к final_suspend
```

Если корутина возвращает `Task<void>`:

```
co_return;
```

то вызывается:

```
promise.return_void()
```

Если в корутине произошло исключение:

```
throw std::runtime_error("oops");
```

то компилятор вызывает:

```
promise.unhandled_exception()
```

15. `co_await`: главная идея

`co_await` — это точка возможной приостановки корутины.

Например:

```
auto x = co_await SomeAsyncOperation();
```

Смысл:

```
если результат уже готов:
    продолжить сразу

если результат не готов:
    приостановить корутину
    передать её handle awaiter-y/runtime-y
    потом кто-то вызовет resume
    после resume вернуть результат из awaiter
```

Важно:

`co_await` не обязательно означает “ждать поток”.
Это не blocking wait.
Это “здесь корутина может приостановиться”.

16. Awaiter

Объект, который стоит справа от `co_await`, должен превратиться в awaiter.

Awaiter должен иметь три метода:

```
bool await_ready();

void await_suspend(std::coroutine_handle<> h);

T await_resume();
```

Или `await_suspend` может возвращать не только `void`, но для базового ответа достаточно `void`.

17. `await_ready`

```
bool await_ready();
```

Смысл:

готов ли результат прямо сейчас?

Если:

```
await_ready() == true
```

то корутина не приостанавливается.

Она сразу вызывает:

```
await_resume()
```

Если:

```
await_ready() == false
```

то корутина будет приостановлена, и дальше вызывается `await_suspend`.

18. `await_suspend`

```
void await_suspend(std::coroutine_handle<> h);
```

Смысл:

корутина сейчас приостанавливается
вот её handle
сохрани его или передай runtime-у

Например, для асинхронного чтения:

```
void await_suspend(std::coroutine_handle<> h) {  
    event_loop.register_read(fd, h);  
}
```

То есть:

```
корутина остановилась  
event loop запомнил: когда fd будет готов, надо вызвать h.resume()
```

19. `await_resume`

```
T await_resume();
```

Вызывается, когда корутина продолжилась после ожидания.

Смысл:

```
вернуть результат выражения co_await  
или бросить исключение
```

Например:

```
int x = co_await ReadIntAsync();
```

После resume вызовется:

```
awaiter.await_resume()
```

и его результат станет значением `x`.

Если операция завершилась ошибкой, `await_resume` может бросить исключение.

20. Полная схема `co_await`

Выражение:

```
auto result = co_await awaitable;
```

примерно раскрывается так:

```
awaiter = getawaiter(awaitable)  
  
if (awaiter.await_ready()) {
```

```

    result = awaiter.await_resume()
} else {
    suspend coroutine
    awaiter.await_suspend(coroutine_handle)
    // управление возвращается вызывающему/runtime-y
    // позже кто-то делает coroutine_handle.resume()
    result = awaiter.await_resume()
}

```

Очень коротко:

```

awaiter_ready   – надо ли реально suspend?
awaiter_suspend – что сделать при suspend?
awaiter_resume  – что вернуть после resume?

```

21. Пример awaiter для event loop

Упрощённо:

```

struct ReadAwaiter {
    EventLoop& loop;
    int fd;
    std::string result;
    std::exception_ptr error;

    bool await_ready() {
        return false;
    }

    void await_suspend(std::coroutine_handle<> h) {
        loop.wait_read(fd, h, &result, &error);
    }

    std::string await_resume() {
        if (error) {
            std::rethrow_exception(error);
        }

        return std::move(result);
    }
};

```

Использование:

```

Task<void> Session(Socket s) {
    while (true) {

```



```

        std::string req = co_await ReadAsync(s);
        std::string resp = Process(req);
        co_await WriteAsync(s, resp);
    }
}

```

Смысл:

ReadAsync возвращает awaitable
 co_await вызывает await_suspend
 event loop получает handle
 fd готов
 event loop resume-ит корутину
 await_resume возвращает req

22. initial_suspend

`initial_suspend` отвечает на вопрос:

Корутина должна начать выполняться сразу или сначала приостановиться?

Например:

```
std::suspend_always initial_suspend();
```

значит:

после создания coroutine frame корутина сразу suspend
 тело ещё не выполняется

А:

```
std::suspend_never initial_suspend();
```

значит:

корутина начинает выполняться сразу

Для ленивых задач часто используют `suspend_always`.

Для “запустить сразу” — `suspend_never`.

23. `final_suspend`

`final_suspend` вызывается в конце корутины.

Зачем он нужен?

дать ожидающей корутине шанс продолжиться
не уничтожать frame мгновенно
передать управление continuation

Например, если одна корутина ждала другую:

```
auto x = co_await ChildTask();
```

то когда `ChildTask` завершилась, в `final_suspend` можно возобновить parent coroutine.

Поэтому `final_suspend` почти всегда должен быть suspend point.

24. Чем C++ coroutine похожа на future

`Task<T>` похож на future:

результат появится позже
можно дождаться через `co_await`
может содержать value или exception

Но C++ coroutine даёт более естественный control flow:

```
Task<int> Foo() {  
    int a = co_await A();  
    int b = co_await B(a);  
    co_return a + b;  
}
```

Вместо:

```
A().ThenValue([](int a) {  
    return B(a).ThenValue([a](int b) {  
        return a + b;  
    });  
});
```

То есть:

```
future chaining:  
    data flow через ThenValue  
  
coroutine:  
    обычный control flow через co_await
```

25. Сходство корутин с файберами

Общее:

```
и файбер, и корутина могут suspend/resume  
оба позволяют писать асинхронный код в последовательном стиле  
оба требуют runtime/scheduler/event loop для реального async I/O  
оба хранят состояние между suspend points
```

Пример:

```
auto req = Read();  
auto resp = Process(req);  
Write(resp);
```

можно реализовать и файберами, и корутинами.

26. Отличие C++ coroutines от файберов

Файбер:

```
имеет свой стек  
может suspend глубоко внутри обычной функции  
переключается через context switch  
дороже по памяти  
больше похож на user-space thread
```

C++ coroutine:

```
stackless  
состояние хранит в coroutine frame  
suspend только в точках co_await/co_yield/co_return  
компилятор генерирует state machine  
обычно дешевле по памяти
```

Главная формула:

```
fiber:
    runtime сохраняет стек

C++ coroutine:
    compiler сохраняет состояние
```

27. Почему “stateless” иногда говорят, но точнее “stackless”

В формулировках иногда случайно говорят “stateless coroutine”, но корректный термин — **stackless coroutine**.

Почему не stateless?

Потому что состояние у корутины как раз есть:

```
state number
локальные переменные
promise object
awaiter
```

Она не stateless.

Она **stackless**, то есть не хранит отдельный полный стек.

Правильная фраза:

```
C++20 coroutines are stackless, not stateless.
```

28. Типичные ошибки

Ошибка 1. Думать, что `co_await` блокирует поток

Нет.

`co_await` может приостановить корутину, но поток ОС может продолжить выполнять другие задачи.

Ошибка 2. Путать `promise_type` и `std::promise`

`promise_type` — часть механизма C++ coroutine.

`std::promise` — синхронизационный примитив для `std::future`.

Названия похожи, смысл разный.

Ошибка 3. Думать, что `stackless coroutine` хранит весь стек

Нет.

Она хранит `coroutine frame`, а не весь `call stack`.

Ошибка 4. Думать, что корутина сама планирует себя

Нет.

Корутина может приостановиться и дать `handle`.

Но кто-то должен потом вызвать:

```
handle.resume();
```

Это делает `event loop`, `scheduler`, `executor` или другая корутина.

Ошибка 5. Уничтожить `coroutine frame` слишком рано

Если корутина `suspend`-нулась, её `frame` должен жить до `resume/destroy`.

Иначе `handle` будет указывать в освобождённую память.

29. Что сказать коротко

Корутина — это обобщение обычной функции: она может приостановиться в середине и потом продолжиться с того же места. `Stackful`-корутина имеет свой стек, поэтому может остановиться глубоко внутри цепочки вызовов; `файбер` — это фактически `stackful`-корутина плюс `scheduler/runtime`. `Stackless`-корутина не имеет отдельного полного стека: компилятор превращает `coroutine`-функцию в `state machine`, а локальные переменные, которые должны пережить `suspend`, выносит в `coroutine frame`.

`C++20 coroutines` — `stackless`. Функция становится корутиной, если в ней есть `co_await`, `co_yield` или `co_return`. При вызове создаётся `coroutine frame`, внутри него `promise object`, состояние автомата, локальные переменные и `awaiter`-ы. Возвращаемый объект, например `Task<T>`, обычно хранит `coroutine_handle`.

`co_await` — это точка возможной приостановки. `Awaiter` имеет методы `await_ready`, `await_suspend`, `await_resume`. Если `await_ready` возвращает `true`, корутина не останавливается. Если `false`, корутина `suspend`-ится, в `await_suspend` передаётся её `handle`, и

runtime может сохранить его. Когда операция завершится, runtime вызовет `handle.resume()`, а результат выражения `co_await` будет получен через `await_resume`.

`co_return` завершает корутину: для значения вызывает `promise.return_value(value)`, для `void` — `promise.return_void()`, после чего корутина проходит через `final_suspend`. Исключения попадают в `promise.unhandled_exception()`.

Главное отличие от фибров: фибер сохраняет стек целиком и переключается runtime-ом через context switch, а C++ coroutine сохраняет только frame одной coroutine-функции и управляется компиляторно с помощью `co_await`.

30. Мини-проверка

1. Что такое корутина?
2. Чем stackful-корутина отличается от stackless-корутины?
3. Почему фибер можно считать stackful-корутиной с runtime?
4. Почему C++20 coroutine правильнее называть stackless, а не stateless?
5. Что такое coroutine frame?
6. Что такое `std::coroutine_handle`?
7. Что такое `promise_type` и почему это не `std::promise`?
8. Что делает `co_await`?
9. Какие три метода есть у awaiter-а?
10. Что делают `await_ready`, `await_suspend`, `await_resume`?
11. Что делает `co_return`?
12. Чем C++ coroutine отличается от фибера?